React Quick Notes

1. What is React?

- A **JavaScript library** for building UI
- Developed by Meta
- Uses **component-based architecture**
- Follows **Virtual DOM** for performance

2. Key Features

- Reusable Components
- Virtual DOM (fast updates)
- One-way Data Binding
- JSX (HTML inside JS)
- Hooks (state + lifecycle in functional components)
  - –spl Functions

    useState, useEffect,useMemo,useCallBack,useForm, useNavigate,useParams,useContext ..etc

3. JSX (JavaScript XML)

- Looks like HTML but inside JS
- Must return **single parent element**

```
const element = <h1>Hello React</h1>;
```

4. Components

Functional Component (Modern)

```
function Welcome() {
  return <h1>Hello</h1>;
}
```

Class Component (Old)

```
class Welcome extends React.Component {
  render() {
    return <h1>Hello</h1>;
  }
```

```
}
```

5. Props (Read-only Data)

Passed from parent → child

```
<Welcome name=”Sanjay”/>
```

```
function Welcome(props) {
  return <h1>Hello {props.name}</h1>;
}
```

6. State (Mutable Data)

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>
      {count}
    </button>
  );
}
```

7. Event Handling

Similar to JS but camelCase

```
<button onClick={handleClick}>Click</button>
<button onClick={ ()=> handleClick(‘arg’) }>Click</button> //with arg
```

8. Conditional Rendering

```
{isLoggedIn ? <Dashboard /> : <Login />}
```

9. Lists & Keys

```
const items = [1,2,3];


items.map(item => (
  <li key={item}>{item}</li>
));
```

10. Forms (Controlled Components)

```
const [name, setName] = useState("");


<input value={name} onChange={(e) => setName(e.target.value)} />
```

uncontrolled forms is not managed by the react state

useRef

hook form is uncontrolled

11. Hooks

useState

- Manage state

useEffect

- Side effects (API, lifecycle)

```
useEffect(() => {
  console.log("Mounted");
}, []);
```

useContext

- Global state

useRef

- DOM reference / persist values

12. Lifecycle (Hooks Version)

Phase	Hook
Mount	useEffect(() => {}, [])
Update	useEffect(() => {}, [dep])
Unmount	useEffect(()=>{ return ()=>{} }) return cleanup function

13. Routing

Using React Router

```
<Route path="/home" element={<Home />} />
```

14. Lifting State Up

- Move shared state to common parent

15. Context API

- Avoid prop drilling

```
const MyContext = createContext();
```

16. Performance Optimization

- React.memo() → avoid child rerendering //Parent having a state called Count, when count changes. It unnecessary rerender child also ….
- useMemo() -> caching values,
- useCallback() -> caching functions

17. API Calls

```
useEffect(() => {
  fetch("API_URL")
    .then(res => res.json())
```

```
  .then(data => console.log(data));
}, []);
```

Note : fetch() returns promise and res.json() returns another promise

React = **Components + State + Props + Hooks**

TanStack Query (React Query)

Current Problem:

Clicked Get Employees – fetch() connect with api returns all emps.

U move away to diff component

And come back again and again clicked getEmps again unnecessary it connect with back end

What it solves

- API fetching + caching + sync + background updates

Core Concepts

- `useQuery` → GET data
- `useMutation` → POST/PUT/DELETE
- Query Keys → caching identity

```
const { data, isLoading } = useQuery({
  queryKey: ['emps'],
  queryFn: getEmployees
})
```

Features

- Caching
- Refetching (auto/manual)
- Background sync
- Pagination & infinite scroll

Important Options

- `staleTime` -how long data is fresh
- `cacheTime` -how long data is in cache
- `refetchOnWindowFocus`

Unit Testing

Definition

Test **smallest unit (function/component)** in isolation

Tools

- (Jest)
- (React Testing Library)

```
test('renders title', () => {

  render(<App />)

  expect(screen.getByText('Hello')).toBeInTheDocument()

})
```

Integration Testing

Definition

Test **multiple units working together**

Example

- Component + API + state

```
render(<EmployeeList />)
```

```
await waitFor(() => screen.getByText('John'))
```

Testing Hooks

```
const { result } = renderHook(() => useCounter())

act(() => {
  result.current.increment()
})

expect(result.current.count).toBe(1)
```

Next.js Quick Notes

(Next.js)

What it is

- React framework with **SSR + SSG + Routing**

Key Features

- File-based routing (`app/` or `pages/`)
- Server & Client components
- API routes
- Built-in optimization

Server vs Client Component

- Server → default, faster, no browser APIs
- Client → `"use client"` needed

Next.js Middleware

- Runs before request
- Used for auth, redirects

```
export function middleware(req) {

  if (!req.cookies.get('user')) {

    return NextResponse.redirect('/login')

  }

}
```